



GUÍA DE ESTUDIO INTEGRADORA - PARCIAL 1

UNIDAD 1: Ingeniería de Software en Contexto

1. Introducción a la Ingeniería del Software

- **Software:** Conjunto de programas, archivos de configuración, documentos de usuario/sistema y herramientas asociadas necesarios para definir, desarrollar y mantener código ejecutable. También se define como información o conocimiento presentado en diferentes niveles de abstracción.
- **Ingeniería de Software (ISW):** Disciplina que abarca todos los aspectos de la producción de software, desde la especificación inicial hasta el mantenimiento post-operativo.
- **Importancia:** Necesidad de producir software confiable de forma rápida y económica. A largo plazo reduce costos, ya que la mayor parte del gasto ocurre en el mantenimiento y evolución post-operativa.
- **Principios:** Entender el problema/negocio antes de proponer soluciones; el diseño correcto aporta calidad y facilitación de mantenimiento.

2. Estado Actual y Antecedentes: La Crisis del Software

- **Origen:** Término introducido en la Conferencia de la OTAN (1968) por Friedrich Bauer (y mencionado por Dijkstra) para describir la dificultad de generar software libre de defectos, comprensible y verificable.
- **Causas Principales:**
 - **Evolución del Hardware:** El avance en circuitos integrados permitió crear hardware más complejo, pero el desarrollo de software (actividad humana e intangible) no avanzó al mismo ritmo.
 - Demanda creciente de sistemas complejos y subestimación de la complejidad intrínseca del software.
 - Falta de metodologías y disciplinas para gestionar el cambio y la producción.
- **Factores Actuales de Fracaso:**
 - *Demandas crecientes:* Se exige "espontaneidad" en la entrega que los métodos tradicionales no cubren.
 - *Bajas expectativas:* Empresas que no aplican ISW entregan productos costosos y poco confiables, conformándose con que "solo funcione".

3. Disciplinas que Conforman la Ingeniería de Software

- **Técnicas (Producto):** Toma de requerimientos, análisis, diseño, implementación, pruebas (testing), despliegue, documentación y capacitación.
- **De Gestión (Proyecto):** Planificación, monitoreo y control de proyectos (ej. metodologías Tradicionales, Lean, Agile).
- **De Soporte (Transversales):** Gestión de Configuración de Software (SCM), Toma de métricas y Aseguramiento de Calidad (QA).

4. Ciclos de Vida (Modelos de Proceso) y la Administración de Proyectos

- **Ciclo de Vida (CV):** Representación simplificada de un proceso que define sus elementos (fases, actividades, productos de trabajo) y el flujo entre ellos.
- **Relación CV Proyecto vs. CV Producto:**
 - El CV del **Producto** es más extenso; dura desde la concepción/idea hasta su retiro del mercado.
 - Un CV de **Producto** puede contener múltiples CV de **Proyectos** (ej. desarrollo inicial, actualizaciones, mejoras).
- **Influencia en la Gestión:** La elección del CV condiciona la planificación y la gestión del proyecto. Ambientes con requerimientos volátiles requieren ciclos iterativos; ambientes estables permiten modelos lineales/secuenciales.

5. Clasificación, Ventajas, Desventajas y Criterios de Elección de Ciclos de Vida

- **Secuencial / Cascada:**
 - *Características:* Avanza linealmente fase a fase. Exige requerimientos congelados al inicio. Dirigido por documentación.
 - *Ventajas:* Estructura clara, minimiza gastos generales de planificación por adelantado. Ideal para equipos técnicamente débiles.
 - *Desventajas:* No tolera incertidumbre o cambios. Producto funcional recién al final. Rediseño costoso si hay fallas.
 - *Criterio de Selección:* Requerimientos muy claros, estables y exhaustivos. Poca incertidumbre.
- **Iterativo / Incremental:**

- *Características:* Desarrolla el sistema en versiones/incrementos. Vincula especificación, desarrollo y validación.
- *Ventajas:* Reduce costos ante cambios. Entrega rápida de valor al cliente. Cliente participa en el proceso.
- *Desventajas:* Proceso menos visible. Costo de documentar cada incremento. Degradación de la estructura del software.
- *Criterio de Selección:* Requerimientos cambiantes o incompletos. Necesidad de entregas parciales y rápidas de valor.

- **Recursivo / Espiral:**

- *Características:* Enfocado en la evaluación y gestión constante de riesgos. Subdivide en mini proyectos.
- *Ventajas:* Excelente gestión de riesgos en proyectos grandes. Permite evaluar factibilidad.
- *Desventajas:* Costoso en tiempo/dinero. La evaluación de riesgos puede ser más cara que el desarrollo.
- *Criterio de Selección:* Proyectos grandes, complejos, con alto nivel de riesgo o requerimientos inciertos.

6. Procesos de Desarrollo: Empíricos vs. Definidos

- **Proceso Definido:**

- *Características:* Determinista (a misma entrada, se espera misma salida). Inspirado en líneas de producción industrial.
- *Enfoque:* Busca la repetibilidad y previsibilidad (costos, tiempos, riesgos). La documentación y el control a menudo priman sobre el producto.
- *Uso:* Admite cualquier ciclo de vida (común en Cascada).

- **Proceso Empírico:**

- *Características:* No determinista. Se basa en la experiencia, la creatividad y la adaptación según el contexto.
- *3 Pilares:* Transparencia, Inspección y Adaptación.

- *Enfoque:* Ciclos cortos de retroalimentación.
- *Uso:* Se combina con ciclos iterativos-incrementales (ej. Scrum, Kanban). Prohibido con Cascada pura.

7. Componentes de un Proyecto de Sistemas de Información

- **Proyecto:** Esfuerzo temporal para crear un producto, servicio o resultado único. Es único, orientado a objetivos y de duración limitada.
- **La Triple Restricción Tradicional:**
 - **Alcance (Fijo):** Requerimientos y funcionalidades iniciales.
 - **Tiempo y Costo (Variables/Ajustables):** Derivan directamente de la estimación del alcance.
 - *Nota:* La calidad no se negocia; romper el equilibrio afecta la calidad final.
- **Líder de Proyecto vs. Equipo:**
 - *Líder:* Toma decisiones, gestiona stakeholders, define plan de proyecto y equilibra la triple restricción.
 - *Equipo:* Cohesivo, multidisciplinario, con sentido de responsabilidad compartida.
- **Stakeholders:** Todos los interesados o involucrados en el proyecto (equipo, cliente, patrocinadores, gerencias).
- **Plan de Proyecto (Elementos claves):** Definición de alcance, ciclo de vida, estimaciones (tamaño, esfuerzo, calendario, costo), gestión de riesgos (identificación, análisis de exposición $ER = P \times I$, mitigación y contingencia), asignación de recursos, programación (Gantt), métricas y controles.

8. Vínculo Proceso - Proyecto - Producto

- **Relación:** El **Proceso** (definición teórica automatizada con herramientas) se adapta al **Proyecto** (donde se incorporan las personas/recursos), el cual obtiene como resultado el **Producto**.
- **Eje Humano:** Dado que el desarrollo de software es una actividad *humano-intensiva*, el éxito depende del equipo capacitado e integrado, más allá de la perfección del proceso formal.

9. Paper: "No Silver Bullet" (Fred Brooks)

- **Tesis Principal:** No existe ninguna innovación tecnológica o de gestión aislada ("bala de plata") que por sí sola prometa mejorar la productividad, simplicidad o fiabilidad del software en un orden de magnitud (10x) en una década.
- **Dificultades Esenciales (Inherentemente ligadas a la naturaleza del software):**
 - *Complejidad:* Crece exponencialmente con el tamaño.
 - *Conformidad:* Debe adaptarse a entornos, interfaces y reglas de negocio cambiantes no diseñadas para el software.
 - *Mutabilidad:* El software sufre cambios continuos en lugar de ser reemplazado.
 - *Invisibilidad:* No tiene representación espacial/geométrica tangible.
- **Dificultades Accidentales (Resueltas históricamente):**
 - Lenguajes de alto nivel, tiempo compartido, entornos de desarrollo unificados (IDEs), orientación a objetos.
- **Soluciones para las Dificultades Esenciales:**
 - **Comprar antes que construir:** Utilizar software existente amortizado en el mercado.
 - **Refinamiento de requerimientos y prototipado rápido:** Descubrir las necesidades reales del usuario iterativamente.
 - **Desarrollo incremental:** Hacer crecer el software orgánicamente en lugar de construirlo todo de una vez.
 - **Cultivar grandes diseñadores/personas capacitadas:** Fomentar y recompensar el talento y la creatividad del equipo.

UNIDAD 2: Gestión Lean-Agile de Productos de Software

1. Filosofía Ágil y Lean

Manifiesto Ágil

- **4 Valores Ágiles:**
 - **Individuos e interacciones** por sobre procesos y herramientas.

- **Software funcionando** por sobre documentación extensiva.
- **Colaboración con el cliente** por sobre negociación contractual.
- **Respuesta ante el cambio** por sobre seguir un plan.
- **12 Principios:** Satisfacción al cliente con entregas tempranas y continuas; aceptar cambios; entregas frecuentes; trabajo diario conjunto negocio-desarrollo; equipos motivados y autoorganizados; comunicación cara a cara; software funcionando como métrica principal; simplicidad (maximizar trabajo no realizado); atención a la excelencia técnica; mejora continua/retrospectivas.
- **Triángulo Ágil (vs. Tradicional):**
 - **Ágil:** Valor y Calidad en el centro. **Tiempo y Costos/Recursos son Fijos**, mientras que el **Alcance es Variable/Estimado**.
 - **Tradicional:** Alcance Fijo, Tiempo y Costo Variables.

Filosofía Lean y sus 7 Principios

Orientada a maximizar el valor al cliente consumiendo el mínimo de recursos y eliminando el desperdicio (*Muda*).

1. **Eliminar Desperdicios:** Todo lo que no agregue valor (funcionalidades extra no usadas, trabajo a medias, procesos extra, esperas, multitasking, defectos, búsqueda de información).
2. **Amplificar el Aprendizaje:** Promover ciclos cortos de feedback e inspección.
3. **Diferir Compromisos (Último momento responsable):** Postergar decisiones irreversibles hasta contar con datos reales.
4. **Entregar lo más Rápido Posible:** Ciclos cortos para capturar valor de mercado rápidamente.
5. **Empoderar al Equipo:** Equipos autogestionados y capacitados.
6. **Construir la Calidad/Integridad Conceptual:** Calidad integrada en cada etapa.
7. **Ver el Todo (Visión Holística):** Optimizar la cadena de valor global.

2. Gestión de Producto y Requerimientos Ágiles

- **Fundamentación:** Requerimientos Just in Time (JIT), basados en conversación cara a cara. Se enfoca en construir el "producto correcto" descubriendo necesidades gradualmente.

- **Product Backlog (PBI):** Lista dinámicamente priorizada por el **Product Owner (PO)** según el valor de negocio. Los ítems superiores tienen mayor detalle y prioridad.

3. User Stories (Historias de Usuario)

- **Concepto:** Técnica para capturar requerimientos de usuario a alto nivel de abstracción. Son porciones **verticales** (cruzan UI, Lógica y Datos).
- **Estructura (Las 3 C's):**
 - **Card (Tarjeta):** Formato: *"Como [Rol] quiero [Actividad] para [Valor de Negocio/Beneficio]"*.
 - **Conversation (Conversación):** Discusión verbal continua entre PO y Equipo para detallar el requerimiento.
 - **Confirmation (Confirmación):** Criterios de Aceptación y Pruebas de Aceptación que validan la implementación.
- **Modelo INVEST (Criterio de Calidad de una US):**
 - Independent (Independiente)
 - Negotiable (Negociable)
 - Valuable (Valiosa para el cliente/negocio)
 - Estimable (Estimable en tamaño)
 - Small (Pequeña, que quepa en un Sprint)
 - Testable (Verificable mediante pruebas)
- **Definiciones clave:**
 - **Definition of Ready (DoR):** Criterios para determinar que una US está lista para ingresar al Sprint (cumple INVEST).
 - **Definition of Done (DoD):** Criterios de calidad que aseguran que el incremento está completamente terminado y desplegable.
- **Niveles de Abstracción:**

- *Temas*: Agrupación conceptual de varias historias.
- *Épicas*: Historias de usuario muy grandes que deben dividirse.
- *User Stories*: Ítems refinados listos para iteración.
- *Spikes*: Investigaciones técnicas o de negocio para reducir incertidumbre.

4. Estimaciones Ágiles

- **¿Qué se estima?:** El **Tamaño relativo** del requerimiento, no el tiempo directo.
- **Unidad de medida: Story Points (Puntos de Historia).** Miden complejidad, esfuerzo e incertidumbre de forma comparativa.
- **Técnica: Planning Poker** (basado en la secuencia de Fibonacci modificada). Busca lograr consenso en el equipo de desarrollo mediante estimación a ciegas.
- **Velocidad:** Cantidad de Story Points que un equipo específico logra completar (DoD) en una iteración/Sprint. Permite predecir la duración del proyecto.

5. Framework Scrum

- **Definición:** Framework ágil, empírico e iterativo-incremental para resolver problemas complejos entregando productos de máximo valor de forma colaborativa.
- **Pilares:** Transparencia, Inspección y Adaptación.
- **Roles:**
 - **Product Owner (PO):** Maximiza el valor del producto y gestiona el Product Backlog.
 - **Scrum Master (SM):** Facilita el framework, remueve impedimentos y promueve la efectividad.
 - **Developers / Equipo de Desarrollo:** Profesionales multifuncionales y autoorganizados que construyen el incremento.
- **Eventos:**
 - *Sprint*: Corazón de Scrum (contenedor de 1 a 4 semanas).
 - *Sprint Planning*: Definición del Sprint Goal y selección de ítems para el Sprint Backlog.

- *Daily Scrum*: Reunión diaria de 15 min para sincronización e inspección del avance hacia el Goal.
- *Sprint Review*: Inspección del Incremento junto a stakeholders y adaptación del Product Backlog.
- *Sprint Retrospective*: Inspección del equipo sobre personas, relaciones, procesos y herramientas para la mejora continua.
- **Artefactos**: Product Backlog, Sprint Backlog e Incremento (con sus respectivos compromisos: Product Goal, Sprint Goal y Definition of Done).

UNIDAD 3: Gestión de la Configuración del Software (SCM)

1. Conceptos Fundamentales

- **SCM (Software Configuration Management)**: Disciplina de soporte transversal que aplica dirección técnica y administrativa para identificar, documentar, controlar y auditar la evolución de las características funcionales y físicas de los ítems de configuración.
- **Problema que resuelve**: Caos en el desarrollo concurrente, pérdida de versiones, falta de trazabilidad, cambios no autorizados y falta de control en los entregables.

2. Elementos Clave de SCM

- **Ítem de Configuración (CI - Configuration Item)**: Pieza de hardware, software o documentación que se somete a control formal de configuración (ej. código fuente, manuales, ERS, casos de prueba).
- **Versión, Variante y Release**:
 - *Versión*: Estado de un CI en un instante de tiempo.
 - *Variante*: Versión alternativa que coexiste para adaptarse a un entorno específico (ej. versión Android vs. iOS).
 - *Release*: Distribución formal de una configuración revisada y testeada para uso del cliente.
- **Línea Base (Baseline)**: Configuración formalmente revisada y acordada que sirve como punto de partida para desarrollos posteriores y que solo se puede modificar mediante un proceso formal de control de cambios.
- **Repositorio y Ramas (Branches)**: Estructura donde se almacenan los CIs. Las ramas permiten evoluciones paralelas y aislamiento de código.

3. Actividades Principales de SCM

1. **Identificación de CIs:** Seleccionar y definir las estructuras de los elementos bajo control.
2. **Control de Cambios:** Proceso formal de solicitud, evaluación de impacto, aprobación (vía Comité de Control de Cambios / CCB) e implementación.
3. **Informe de Estado (Status Accounting):** Registrar e informar el estado actual de las solicitudes de cambio y baselines.
4. **Auditorías de Configuración:** Verificar que los CIs cumplan con sus requerimientos funcionales y físicos especificados.

4. SCM en Ambientes Ágiles

- Integración con prácticas de entrega continua:
 - **Integración Continua (CI):** Automatización de la compilación y pruebas del código de forma frecuente para detectar fallas rápido.
 - **Entrega Continua (CD - Continuous Delivery):** Automatización de la puesta a disposición del software en entornos de prueba o producción listos para su despliegue.
 - **Despliegue Continuo (CD - Continuous Deployment):** Despliegue automático y directo a producción de cada cambio que pasa las pruebas.
 - Estrategias de Despliegue: *Canary Deployments* (liberación gradual a un subconjunto de usuarios) y *Blue/Green Deployment* (dos entornos idénticos donde uno está en vivo y el otro se actualiza).